AI Engineer – Take-Home Exercise Briefing

Real-World Context: Statista AI Chat

At Statista, we're building an AI-powered chat interface where users ask questions about consumer behavior, market trends, and business intelligence. The AI chat analyzes our data and returns structured insights.

Current user flow:

1. User asks a question: *"What influences Gen Z purchase decisions and what brands are popular?"*
2. AI chat analyzes Statista data and returns a structured response (JSON) with key insights, metrics, and sources
3. **Next step (your task):** Convert that chat response into professional, presentation-ready slides

This is a real problem we need solved: turning data insights into polished presentations automatically.

The Exercise

What You Need to Build

A service that takes AI chat output (as structured JSON) and generates slides for a professional presentation.

Inputs:

- Structured JSON-formatted Statista AI chat answer
  - See `gen_z_purchase_behavior_analysis.json` for a real example
  - Contains: analysis summary, insight categories, metrics, charts, source metadata
- Optional: user preferences (e.g., tone, branding, slide count, specific focus areas)
  - e.g., a user prompt such as “Turn this into a 5-slide presentation to be shared with C-level colleagues - focus on …”

Output:

- Slides that capture the gist of the given chat answer (and the given user prompt)
  - Include charts/visualizations where appropriate
  - Add proper source attribution / citations

Important Ground Rules

You should use whatever tools help you get this done. LLMs, Copilot, libraries you've never used before—look stuff up, get help, iterate. The only thing we care about is that you understand what your code is actually doing.

We'll interview you on this, and we're going to ask straightforward questions:

- "What does this part do?"
- "Why'd you pick that library?"
- "How would you add X feature?"
- "What breaks if the input is malformed?"

If you can answer those, you're good. We're not testing whether you memorized API docs or can code without looking stuff up.

What to Build With

You've got options. Use what you're comfortable with or what makes sense, e.g., you may use **Python** (e.g., using python-pptx), **Java**, or another programming language. For PDFs and PowerPoints there are standard libraries in most languages.

Scope

Plan for maybe **2–4 hours** of actual work. That includes futzing around with libraries, asking an LLM for help, and fixing bugs. The goal is a working prototype, not a production system.

Keep it simple. Don't build a web UI with authentication and a database unless that feels natural to you. A script you run from the command line works just fine.

What to Submit

1. **Code** – put it in a folder or Git repo
2. **README** – quick explanation of:
   - What the thing does
   - How to run it
   - Which libraries you used and why you picked them
   - Anything you know doesn't work or would improve with more time
3. **Example output** – run it once and show us what the PDF/PowerPoint looks like

---

Interview

When we talk about this, expect stuff like:

- Walk through the code. What's happening here?
- Why that approach instead of another one?
- What if the user gave you [weird edge case]? How'd you handle it?
- What would you do differently if you had more time?

We won't ask you to:

- Memorize library APIs
- Code something from scratch in real-time without being able to reference things
- Write production-quality code with full test coverage

What We're Actually Judging

- Does it work?
- Can you explain your choices?
- Is the code understandable, even if it's not perfect?
- Did you think through edge cases or limitations?
- ...

That's it. We want to hire people who can build things and understand them. The code doesn't have to be perfect.

Hit us up if something's unclear about what we're asking for.